

Customer Identity Resolution cho dữ liệu khách hàng đa nguồn

Thiết kế và vận hành dựa trên schema và mã nguồn hiện tại

2026-08-04

Customer Identity Resolution cho dữ liệu khách hàng đa nguồn

Tóm tắt

Customer Identity Resolution (CIR) là quá trình liên kết các bản ghi khách hàng từ nhiều nguồn thành một hồ sơ thống nhất. Trong hệ thống này, dữ liệu đầu vào được lưu vào bảng staging, sau đó được xử lý bởi resolver Python theo các quy tắc metadata. Mục tiêu là tạo ra một master profile duy nhất cho mỗi khách hàng trong từng tenant và domain, đồng thời giữ được tính linh hoạt khi dữ liệu mới hoặc quy tắc matching thay đổi.

1. Phạm vi

Tài liệu này dựa trên các thành phần hiện có trong repository:

- schema PostgreSQL trong database-init/database-schema.sql
- module resolver trong backend-system/identity_resolution/identity_resolution/resolver.py
- trigger controller trong backend-system/identity_resolution/identity_resolution/trigger_controller.py
- logic persona trong backend-system/identity_resolution/identity_resolution/persona.py

2. Mô hình dữ liệu chính và Cơ cấu làm giàu dữ liệu (Data Enrichment)

Cơ sở dữ liệu lưu trữ các thực thể và mối quan hệ để phục vụ quá trình liên kết thông tin. Dưới đây là chi tiết các bảng cốt lõi cùng các trường dữ liệu quan trọng, ý nghĩa vận hành, và cách thức làm giàu dữ liệu (data enrichment):

2.1 Bảng cdp_raw_profiles_stage (Bảng staging dữ liệu thô)

Bảng trung gian đóng vai trò là landing zone để tiếp nhận vết thông tin khách hàng thô được đẩy vào từ các nguồn khác nhau trước khi giải quyết danh tính.

- **Các trường dữ liệu quan trọng:**
 - raw_profile_id (UUID, Khóa chính): Định danh duy nhất cho mỗi sự kiện raw profile.
 - tenant_id (UUID): Phân lý để bảo vệ an toàn dữ liệu multi-tenant.
 - domain (TEXT): Lĩnh vực nghiệp vụ chuyên biệt (ví dụ: retail, banking, travel).
 - source_system (TEXT): Hệ thống gốc gửi dữ liệu (ví dụ: POS, AppsFlyer, MoEngage, CRM).
 - external_customer_id (TEXT): Mã khách hàng nội bộ của hệ thống nguồn.
 - email, phone_number, national_id (TEXT): Dữ liệu định danh cá nhân (PII). Có thể là plaintext hoặc chuỗi SHA-256 hash một chiều.
 - full_name, first_name, last_name (TEXT): Thông tin họ tên tại nguồn.
 - device_id, advertising_id, cookie_id, push_token (TEXT): Định danh số và thiết bị phục vụ định dạng đa kênh (multi-channel resolution).
 - event_name, event_time (TIMESTAMP WITH TIME ZONE), event_payload (JSONB): Tên hành vi, mốc thời gian và toàn bộ dữ liệu thuộc tính bổ sung dạng bán cấu trúc phục vụ làm giàu thông tin (data enrichment).
 - status_code (SMALLINT, Mặc định = 1): Quản lý vòng đời hàng đợi (1: Sẵn sàng xử lý, 2: Đang xử lý, 3: Đã xử lý hoàn tất, 4: Thất bại).
 - processed_at (TIMESTAMP WITH TIME ZONE): Thời điểm hoàn tất giải quyết danh tính.

2.2 Bảng cdp_master_profiles (Hồ sơ Master / Golden Record)

Bản ghi duy nhất của một khách hàng sau khi đã được hợp nhất, làm sạch và làm giàu thông tin từ mọi nguồn dữ liệu.

- **Các trường dữ liệu quan trọng:**

- master_profile_id (UUID, Khóa chính): Định danh hồ sơ vàng của khách hàng.
- tenant_id (UUID), domain (TEXT): Ràng buộc phạm vi quản trị dữ liệu.
- full_name, first_name, last_name, email, phone_number, national_id, address (TEXT): Thông tin định danh đã được chuẩn hóa (consolidated) theo thứ tự ưu tiên hoặc thời gian.
- secondary_emails, secondary_phones (JSONB): Lưu mọi email/SĐT phụ khác thu thập từ nguồn hàng ngày dưới dạng mảng để không bỏ sót kênh tiếp cận.
- external_ids (JSONB): Bản đồ lưu (key/value) cặp source_system và external_customer_id tương ứng nâng cao khả năng đồng bộ ngược (reverse syndication).
- device_ids, advertising_ids, cookie_ids (TEXT[]): Tập hợp duy nhất các token thiết bị để tiếp cận quảng cáo.
- push_tokens (JSONB): Cấu trúc key-value lưu push token tương thích với các nền tảng thông báo (như FCM, APNS).
- is_hashed (BOOLEAN, Mặc định = FALSE): Đánh dấu nếu hồ sơ này sử dụng các PII bị che giấu bằng hàm hash bảo mật SHA-256.
- persona_name (TEXT): Tên danh tính ảo dạng dễ đọc (non-PII) được tự động sinh thông qua logic nghiệp vụ hoặc LLM (Gemini-3.5-Flash) để phục vụ tìm kiếm và bảo mật.
- persona_summary (TEXT): Bản tóm tắt phong cách hành vi được cập nhật từ lịch sử tương tác đa kênh phục vụ tiếp cận cá nhân hóa sâu (data enrichment).
- persona_embedding (VECTOR(768)): Vector nhúng nhị phân từ LLM hỗ trợ tìm kiếm ngữ nghĩa (semantic search) và đề xuất đối tượng Lookalike.
- updated_at (TIMESTAMP WITH TIME ZONE): Thời điểm cập nhật cuối cùng của hồ sơ.
- first_seen_raw_profile_id (UUID): Truy vết lineage về raw profile ban đầu khai sinh ra hồ sơ này.
- source_systems (TEXT[]): Danh sách phân biệt các nguồn đóng góp dữ liệu.

2.3 Bảng cdp_profile_links (Bảng liên kết quan hệ)

Mô tả bản đồ ánh xạ trạng thái quan hệ 1-N giữa hồ sơ master hoạt động và tất cả các nguồn thô đóng góp.

- **Các trường dữ liệu quan trọng:**

- link_id (BIGSERIAL, Khóa chính): Định danh duy nhất cho bản ghi liên kết.
- tenant_id (UUID): Ràng buộc tenant.
- raw_profile_id (UUID, UNIQUE): Liên kết duy nhất tới staging. Ràng buộc UNIQUE đảm bảo tại một thời điểm, một bản thô chỉ ánh xạ tới duy nhất một hồ sơ master hoạt động.
- master_profile_id (UUID): Tham chiếu tới hồ sơ master đích.
- match_score (NUMERIC): Điểm số đánh giá độ tin cậy trùng khớp từ 0.0 đến 1.0.
- match_method (TEXT): Phương thức/Thuật toán kích hoạt liên kết (ví dụ: exact_email, fuzzy_trgm_name).
- status (VARCHAR): Trạng thái liên kết (ACTIVE, HISTORICAL nếu bị gỡ bỏ sau khi unmerge hồ sơ).

2.4 Bảng cdp_profile_attributes (Metadata thuộc tính)

Registry điều khiển cho phép hệ thống mở rộng và tùy biến linh hoạt quy tắc ghép nối và làm giàu thuộc tính.

- **Các trường dữ liệu quan trọng:**

- attribute_internal_code (VARCHAR, Khóa chính): Khóa nội bộ tương khớp cột ở staging.
- master_profile_column (VARCHAR): Cột đích tương ứng trên bảng master profile.
- is_identity_resolution (BOOLEAN): Đánh dấu trường thuộc tính này có tham gia trực tiếp vào bước tìm kiếm so khớp danh tính hay không.
- matching_rule (VARCHAR): Thuật toán so khớp (exact, fuzzy_trgm, fuzzy_dmetaphone, none).
- matching_threshold (NUMERIC): Mức tối thiểu chấp nhận trùng khớp khi dùng phép so sánh mờ.
- consolidation_rule (VARCHAR): Chiến lược ghi đè, merge hành vi khi cập nhật thuộc tính master (most_recent - lấy mới nhất, verified_first - ưu tiên nguồn uy tín, non_null - giữ nguyên giá trị không rỗng, append_distinct - nối mảng bộ lọc).

2.5 Bảng `cdp_identity_index` (Chỉ mục định danh phẳng)

Bảng tăng tốc tra cứu khớp chính xác định danh bằng cách phẳng hóa các mảng hoặc bản đồ định danh, giúp tránh scan các cột JSONB/mảng phức tạp trong các luồng dữ liệu throughput cao.

- **Các trường dữ liệu quan trọng:**

- `identity_index_id` (UUID, Khóa chính): Định danh chỉ mục định danh phẳng.
 - `tenant_id` (UUID): Scoping theo tenant.
 - `master_profile_id` (UUID): Tham chiếu tới master profile sở hữu định danh này.
 - `identifier_type` (VARCHAR): Loại mã định danh (ví dụ: `email`, `phone`, `cookie_id`).
 - `identifier_value` (TEXT) & `identifier_value_normalized` (TEXT): Giá trị định danh gốc và giá trị định danh đã chuẩn hóa (chuyển chữ thường, cắt khoảng trắng dư) để so khớp chính xác.
 - `is_blocked` (BOOLEAN): Cờ chặn định danh giả lập (ví dụ: `anonymous`, `null`, `void`).
-

3. Cơ chế xử lý

Quá trình giải quyết danh tính Customer Identity Resolution (CIR) được điều phối bởi module `CustomerIdentityResolver` trong `resolver.py`. Hệ thống hoạt động hoàn toàn dựa trên metadata điều khiển (`cdp_profile_attributes`), đảm bảo tính linh hoạt, mở rộng và cách ly dữ liệu đa người dùng (multi-tenant isolation).

3.1 Quy trình xử lý chi tiết (Step-by-Step Processing)

Mỗi đợt xử lý (batch) thực thi theo chuỗi 7 bước tuần tự:

1. **Tải cấu hình quy tắc (Fetch Active Rules):**

- Quét bảng `cdp_profile_attributes` để lấy danh sách các trường được đánh dấu `is_identity_resolution = TRUE`, `status = 'ACTIVE'`, có `matching_rule` khác `none` và không rỗng.

2. **Trích xuất dữ liệu Staging (Fetch Unprocessed Profiles):**

- Quét bảng `cdp_raw_profiles_stage` lấy ra các bản ghi mới chèn chưa qua xử lý (`status_code = 1`) giới hạn theo kích thước `batch_size` (mặc định 1,000 bản ghi).

3. **Thiết lập ngữ cảnh an toàn Tenant (Set Tenant Context):**

- Đối với mỗi bản ghi thô, hệ thống thực thi `SELECT set_config('app.tenant_id', ...)` để kích hoạt cơ chế Row-Level Security (RLS) của PostgreSQL, đảm bảo tuyệt đối không rò rỉ dữ liệu giữa các tenant.

4. **Xây dựng truy vấn so khớp động (Dynamic Query Building):**

- Dựa vào danh sách rule, hệ thống kiểm tra các thuộc tính có trong bản ghi thô để xây dựng câu lệnh SQL OR động:
 - **Mảng thiết bị (`device_id`, `advertising_id`, `cookie_id`):** Dùng toán tử `= ANY(array_column)`.
 - **Định danh nguồn (`external_customer_id`):** Dùng toán tử chứa JSONB `external_ids @> jsonb_build_object(source_system, value)`.
 - **Thuộc tính khớp chính xác (`exact`):** Dùng toán tử `=`.
 - **Khớp mờ chuỗi (`fuzzy_trgm`):** Dùng hàm `similarity(column, value) >= threshold`.
 - **Khớp ngữ âm (`fuzzy_dmetaphone`):** Dùng hàm `dmetaphone(column) = dmetaphone(value)`.

5. **So khớp Master Profile (Find Master Profile):**

- Thực thi câu lệnh SQL kết hợp điều kiện phân vùng bắt buộc: `WHERE tenant_id = :tenant_id AND domain = :domain AND (các_điều_kiện_động) LIMIT 1`.

6. **Hợp nhất dữ liệu hoặc Khai sinh Hồ sơ Vàng (Merge or Create):**

- **Nếu TÌM THẤY Master Profile:**

- Tạo bản ghi liên kết mới trên `cdp_profile_links` với `match_score = 1.0` và `match_method = 'DynamicMatch'`.
- Hợp nhất các trường thuộc tính scalar (họ tên, email, SDT, v.v.) theo chiến lược cấu hình `consolidation_rule` (`most_recent`, `verified_first`, `source_priority`, `non_null`, `append_distinct`, `overwrite`) hoặc `COALESCE` mặc định.
- Tích lũy (`append/union`) các mảng thiết bị, mã nguồn `source_systems` và các bản đồ JSONB (`external_ids`, `push_tokens`, `communication_preferences`).
- Kiểm tra định dạng PII: nếu bị hash, bật cờ `is_hashed = TRUE` và tự động sinh `persona_name`.

- **Nếu KHÔNG TÌM THẤY Master Profile:**

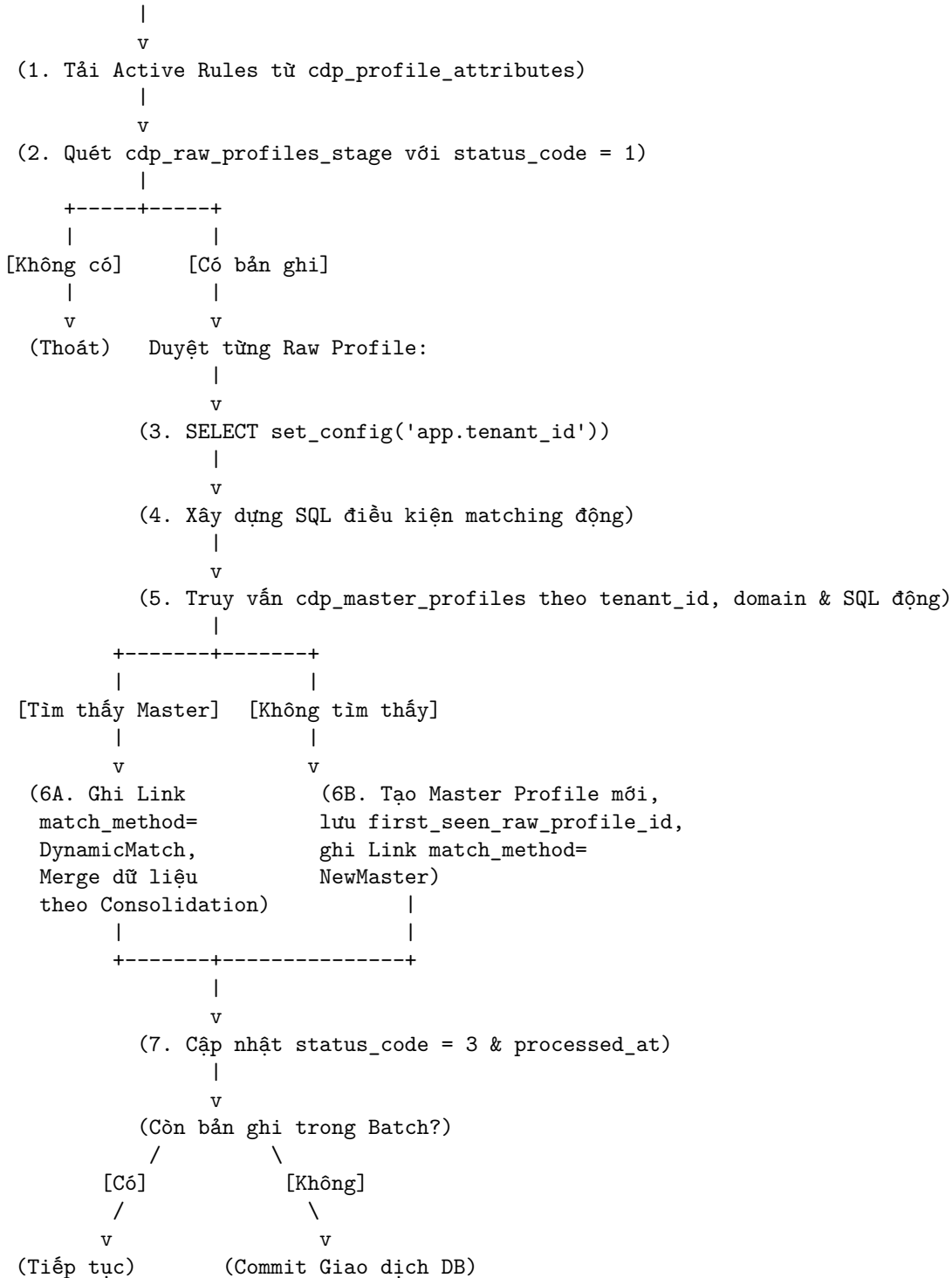
- Khởi tạo một Master Profile mới trên `cdp_master_profiles`, lưu vết `first_seen_raw_profile_id`.
- Tạo bản ghi liên kết đầu tiên trên `cdp_profile_links` với `match_method = 'NewMaster'`.

7. Cập nhật trạng thái Staging & Commit (Mark Processed & Commit):

- Cập nhật bản ghi staging thành `status_code = 3` (hoàn tất) và ghi mốc thời gian `processed_at = NOW()`.
- Sau khi duyệt hết batch, thực thi `conn.commit()` để hoàn tất giao dịch atomic. Nếu gặp sự cố, thực hiện `conn.rollback()`.

3.2 Sơ đồ luồng xử lý tổng thể (Resolution Execution Flow)

[Bắt đầu Batch Resolution]



3.3 Quy tắc matching và ví dụ dữ liệu

Hệ thống hỗ trợ 4 cơ chế matching chính được cấu hình động qua thuộc tính `matching_rule` trong `cdp_profile_attributes`. Dưới đây là mô tả chi tiết kèm theo ví dụ dữ liệu thực tế:

1. Quy tắc exact (Khớp chính xác) Áp dụng cho các định danh có tính duy nhất cao và có cấu trúc ổn định. Hệ thống thực hiện so sánh bằng toán tử = hoặc cơ chế kiểm tra phần tử trong mảng (containment) của PostgreSQL cho trường mảng (như email hoặc phone_number lưu dạng danh sách).

• **Ví dụ dữ liệu:**

- **Inbound Raw Profile (Staging):**

- * email: "nguyena@gmail.com"
- * phone_number: "+84901234567"

- **Existing Master Profile (Master):**

- * email: "nguyena@gmail.com"
- * phone_number: "+84901234567"

- **Kết quả:** Hệ thống tìm thấy trùng khớp hoàn toàn giá trị email và liên kết bản ghi mới vào Master Profile hiện có này.

2. Quy tắc fuzzy_trgm (Khớp mờ theo Trigram) Áp dụng cho các trường dữ liệu dạng chuỗi văn bản dễ sai lệch nhỏ như địa chỉ, tên tổ chức hoặc họ tên đầy đủ. Quy tắc này sử dụng extension pg_trgm để tính toán độ tương đồng dựa trên số lượng cụm 3 ký tự (trigrams) chung. Ngưỡng chấp nhận được kiểm tra qua matching_threshold (thường mặc định từ 0.6 trở lên).

• **Ví dụ dữ liệu:**

- **Inbound Raw Profile (Staging):**

- * full_name: "Nguyễn Văn A" (không dấu hoặc gõ sai: "Nguyen Van A")
- * address: "123 Đường Lê Lợi, Phường 1, Quận 1, TPHCM"

- **Existing Master Profile (Master):**

- * full_name: "Nguyễn Văn A"
- * address: "123 Lê Lợi, P.1, Q.1, TP. HCM"

- **Kết quả:** Nhờ phép tính mờ Trigram trên địa chỉ hoặc họ tên gõ lệch nhẹ, độ trùng khớp vượt ngưỡng matching_threshold = 0.65, hệ thống tự động xác định đây là cùng một người.

3. Quy tắc fuzzy_dmetaphone (Khớp mờ theo ngữ âm Double Metaphone) Áp dụng cho việc so sánh họ tên quốc tế hoặc các ký hiệu không dấu dễ biến âm khi chuyển ngữ. Double Metaphone mã hóa mỗi từ thành một mã ngữ âm đại diện cho cách phát âm của từ đó. Đối chiếu hai mã ngữ âm này giúp ghép nối chính xác bất kể sai lệch chính tả nhỏ.

• **Ví dụ dữ liệu:**

- **Inbound Raw Profile (Staging):**

- * first_name: "Smith"

- **Existing Master Profile (Master):**

- * first_name: "Smyth"

- **Kết quả:** Cả hai từ đều được mã hóa thành mã ngữ âm mã SMO. Kết quả so khớp thành công dù chính tả viết khác nhau.

4. Quy tắc none (Bỏ qua) Bỏ qua thuộc tính này trong việc ghép nối danh tính, thuộc tính chỉ được dùng để bổ sung thông tin bổ trợ (enrichment) sau khi đã giải quyết xong danh tính qua các khóa khác.

4. Xử lý dữ liệu nhạy cảm (Privacy & Hashed PII Handling)

Để tuân thủ các quy định bảo vệ dữ liệu cá nhân (như Nghị định 13/2023/NĐ-CP) và tương thích với mô hình hashed-match trên các nền tảng quảng cáo (Google, Meta, TikTok), hệ thống hỗ trợ cơ chế tiếp nhận PII đã bị băm 1 chiều dạng SHA-256 (64 ký tự hex).

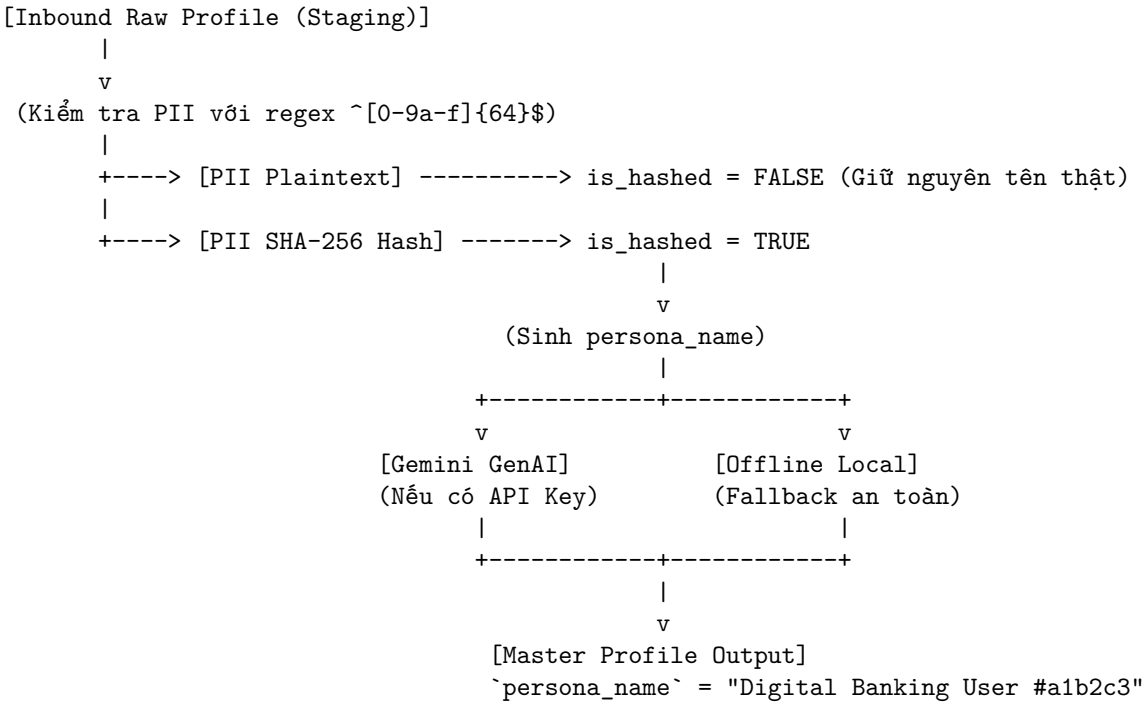
4.1 Quy trình nhận diện và tự động sinh Persona Name

Khi dữ liệu PII (full_name, email, phone_number, national_id) được nạp vào ở dạng SHA-256 hash, hệ thống không thể đảo ngược để hiển thị tên thật. Do đó, tầng ứng dụng (persona.py) tự động kiểm tra định dạng dữ liệu và kích hoạt luồng sinh persona_name (danh tính thay thế non-PII):

- Kiểm tra tự động:** Sử dụng biểu thức chính quy (`^[0-9a-f]{64}$`) để xác định xem PII có bị hash hay không, tự động bật cờ `is_hashed = TRUE` trên `cdp_master_profiles`.
- Sinh danh tính ảo (Persona Generation):**

- **Luồng LLM (Google Gemini):** Nếu cấu hình GOOGLE_GENAI_API_KEY, hệ thống gửi các thuộc tính phi PII (domain, media_source, channel) sang Gemini để tạo tên đại diện gợi nhớ.
- **Luồng Offline Deterministic (Fallback):** Nếu không có internet hoặc API key, hệ thống tính sha256 trên định danh mở neo (device_id, advertising_id, v.v.) để tạo tên định hình cố định kèm hậu tố hash 6 ký tự (ví dụ: Savvy Retail Shopper (TikTok Ads) #4f2a9c).

4.2 Luồng xử lý dữ liệu nhạy cảm (Sequence Flow)



4.3 Ví dụ dữ liệu thực tế (Hashed vs Plaintext)

Trường thuộc tính	Plaintext Inbound	Hashed Inbound (SHA-256)	Master Profile Lưu trữ
full_name	"Nguyen Van An"	9f86d08188...15b0f00a08	9f86d08188...15b0f00a08
email	"an.nguyen@gmail.com"	d081884c7d...c15b0f00a08	d081884c7d...c15b0f00a08
is_hashed	FALSE	TRUE	TRUE
persona_name	NULL	(Tự động sinh)	"Savvy Retail Shopper #4f2a9c"

5. Kích hoạt và Giới hạn Tần suất (Trigger & Throttling)

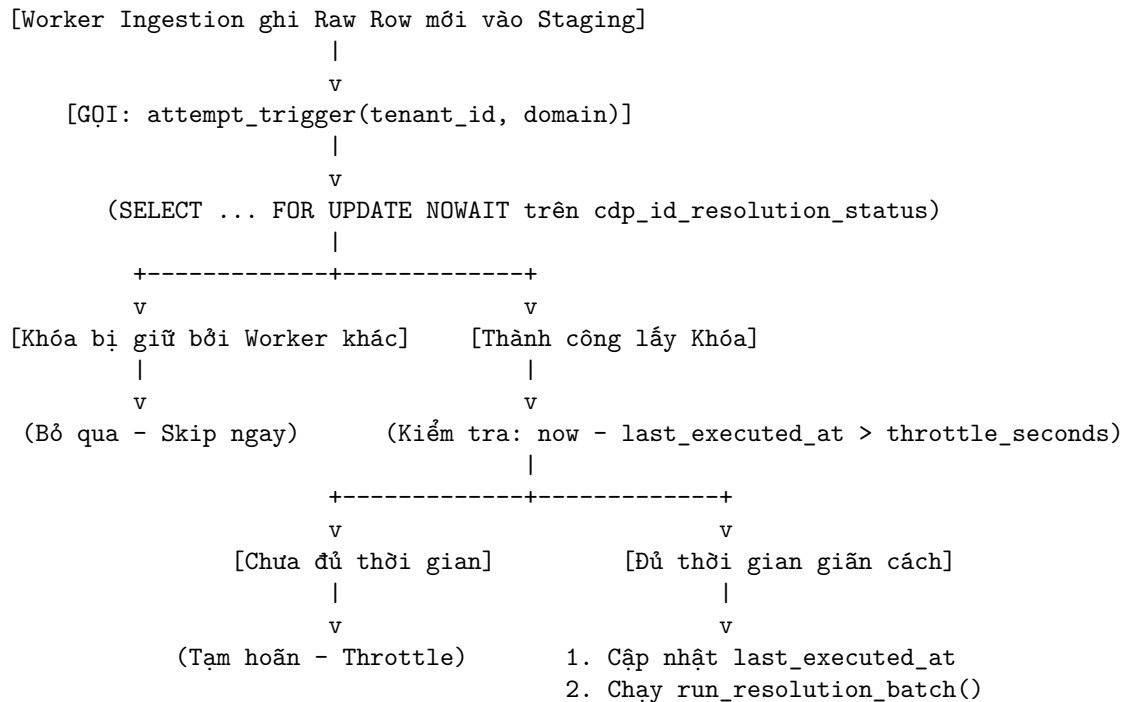
Để đảm bảo hiệu năng trong môi trường streaming lượng lớn bản ghi đầu vào, hệ thống kết hợp cơ chế xử lý **gần thời gian thực (Near Real-time)** và **chạy lô định kỳ (Daily Batch Job)**.

5.1 Cơ chế Throttling khóa hàng chờ (Row-Level Lock)

Hệ thống không sử dụng DB Trigger PL/pgSQL trực tiếp nhằm tránh gây nghẽn (lock contention) trên cơ sở dữ liệu. Thay vào đó, worker phía Ingestion chủ động gọi IdentityResolutionTrigger.attempt_trigger sau mỗi đợt chèn dữ liệu thô mới:

1. Trạng thái chạy được kiểm soát qua bảng trạng thái đơn dòng cdp_id_resolution_status.
2. Sử dụng truy vấn SELECT ... FOR UPDATE NOWAIT để kiểm tra khóa:
 - Nếu bảng đang bị khóa bởi một worker khác, lệnh trigger hiện tại sẽ **ngay lập tức bỏ qua (skip)** mà không làm tắc nghẽn thread.
 - Nếu khoảng thời gian kể từ lần chạy trước nhỏ hơn throttle_seconds (mặc định 5 giây), trigger sẽ tạm hoãn để tích lũy gom lô (micro-batching).
3. Khi đủ điều kiện, worker cập nhật mốc last_executed_at và thực thi hàm run_resolution_batch().

5.2 Luồng điều phối Trigger & Throttling (Control Flow)



5.3 Ví dụ dữ liệu vận hành Bảng Trạng thái

Bảng cdp_id_resolution_status:

id	last_executed_at	updated_at	Trạng thái hệ thống
TRUE	2026-08-04 10:00:00+07	2026-08-04 10:00:00+07	Đã chạy đợt 1 thành công. Lần trigger tại 10:00:02 sẽ bị throttle (do < 5s). Lần trigger tại 10:00:06 sẽ được chấp nhận chạy.

6. Các điểm cần lưu ý khi vận hành (Operational Best Practices)

1. Tính Toàn vẹn và Bất biến (Idempotency):

- Pipeline chỉ quét các bản ghi staging có status_code = 1. Khi xử lý xong, hệ thống chuyển status_code = 3 và đóng mốc processed_at = NOW().
- Khóa duy nhất UNIQUE(tenant_id, raw_profile_id) trên cdp_profile_links đảm bảo việc chạy lại batch (retry) khi gặp sự cố không bao giờ gây nhân bản liên kết hoặc tạo dư thừa Master Profile.

2. Bảo mật và Phân vùng đa khách hàng (Tenant & Domain Scoping):

- Tất cả các câu lệnh SQL tự động sinh bởi Resolver luôn chứa điều kiện bắt buộc WHERE tenant_id = :tenant_id AND domain = :domain. Điều này ngăn chặn triệt để nguy cơ rò rỉ dữ liệu chéo giữa các đơn vị kinh doanh hoặc khách hàng doanh nghiệp khác nhau.

3. Tiên xử lý và Chuẩn hóa Dữ liệu (Data Cleansing):

- Số điện thoại cần được đưa về chuẩn E.164 (ví dụ: +84901234567).
- Email cần chuyển thành chữ thường (lowercase) và cắt bỏ khoảng trắng thừa (trim) trước khi ghi vào staging để đảm bảo tính chính xác cho các quy tắc exact_match.

4. Xử lý sự cố và Giám sát Hàng đợi:

- Cần thiết lập cảnh báo (alerting) khi số lượng bản ghi có status_code = 1 tồn đọng quá ngưỡng trên bảng cdp_raw_profiles_stage hoặc khi xuất hiện các bản ghi lỗi status_code = 4.

7. Kết luận và Đánh giá Giải pháp (Conclusion & Benchmark)

Hệ thống Customer Identity Resolution (CIR) trong kiến trúc Customer 360 này là một pipeline xử lý dữ liệu hiện đại, tenant-aware và hoàn toàn điều khiển bởi metadata (metadata-driven). Kiến trúc cho phép vận hành linh hoạt ở cả hai chế độ gần thời gian thực (Near Real-time với Throttling) và chạy lô định kỳ (Daily Batch), tạo ra Golden Record duy nhất mà không làm nghẽn cơ sở dữ liệu.

7.1 Ma trận Đáp ứng Trường hợp Sử dụng (Use Case Application Matrix)

Kịch bản Nghiệp vụ (Use Case)	Yêu cầu Kỹ thuật chính	Cơ chế Giải quyết trong C360 CIR Engine
1. Hợp nhất Đa kênh (Cross-Channel Stitching)	Liên kết hành vi từ Web Tracking, App, POS, CRM và Ads	Khớp chính xác hoặc mờ trên email/phone, đồng thời tích lũy mảng device_ids, cookie_ids, advertising_ids và JSONB external_ids.
2. Quản trị Đa đơn vị / Đa ngành (Multi-Tenant & Multi-Domain)	Phân vùng dữ liệu an toàn giữa các tenant và domain (Retail, Banking, Real Estate, Travel)	Khóa cứng điều kiện SQL WHERE tenant_id = :tenant_id AND domain = :domain kết hợp cơ chế Row-Level Security (RLS) của PostgreSQL.
3. Xử lý PII Băm Bảo mật (Privacy-Preserving & AdTech Match)	Nhận dữ liệu băm SHA-256 từ Meta/Google/TikTok Ads mà không lộ tên thật	Biểu thức chính quy tự động phát hiện PII băm (^[0-9a-f]{64}\$), bật cờ is_hashed = TRUE, tự động sinh persona_name qua Gemini GenAI / Local Fallback và tạo Vector Embedding (768d).
4. Xử lý Dữ liệu Sai lệch / Nhập sai (Dirty Data & Typo Handling)	Ghép nối khách hàng khi thu ngân POS nhập sai chính tả tên hoặc địa chỉ	Cấu hình quy tắc khớp mờ Trigram (fuzzy_trgm) hoặc ngữ âm (fuzzy_dmetaphone) với ngưỡng tin cậy tùy chỉnh (matching_threshold).
5. Cập nhật Thuộc tính Ưu tiên (Conflict & KYC Resolution)	Xử lý mâu thuẫn khi dữ liệu cũ đề dữ liệu mới hoặc nguồn không tin cậy	Cấu hình chiến lược consolidation_rule linh hoạt: verified_first (ưu tiên bản ghi KYC), most_recent (theo timestamp), source_priority (theo độ uy tín hệ thống nguồn), hoặc append_distinct.

7.2 So sánh Chi tiết với Giải pháp Thương mại (Twilio Segment Unify vs Native C360 CIR)

Dưới đây là ma trận so sánh đối chiếu giữa hệ thống **Native C360 CIR Engine** và giải pháp CDP thương mại phổ biến **Twilio Segment Unify (Personas)**:

Tiêu chí So sánh	Native Customer 360 CIR Engine	Twilio Segment Unify (Personas)
Kiến trúc & Làm chủ Dữ liệu (Deployment & Governance)	Native trên PostgreSQL 16 (On-Premises / Private Cloud). Làm chủ 100% dữ liệu và hạ tầng, không nguy cơ vendor lock-in.	Managed SaaS Cloud. Dữ liệu trung chuyển qua hạ tầng của Segment; tuân thủ chính sách lưu trữ của bên thứ ba.
Thuật toán So khớp (Matching Engine)	Hybrids: Kết hợp Khớp chính xác (Exact), Khớp mờ Trigram (pg_trgm), Khớp ngữ âm Double Metaphone (dmetaphone) qua metadata.	Deterministic Identity Graph: Phụ thuộc chủ yếu vào quy tắc ghép nối cứng qua userId và anonymousId. Hạn chế khớp mờ tự động.

Tiêu chí So sánh	Native Customer 360 CIR Engine	Twilio Segment Unify (Personas)
Bảo mật Multi-Tenant (Multi-Tenant Isolation)	Cách ly triệt để theo cấp CSDL (PostgreSQL Row-Level Security & Tenant Scoping). Hỗ trợ chia sẻ hạ tầng hiệu quả.	Cách ly theo cấp Workspace / Source. Chi phí tăng tiến khi mở rộng nhiều Workspace cho từng tenant độc lập.
Chiến lược Hợp nhất Thuộc tính (Consolidation Strategy)	Cấu hình linh hoạt theo từng trường (most_recent, verified_first, source_priority, append_distinct, overwrite). Tự động phát hiện SHA-256 PII, sinh persona_name bằng LLM (Gemini 3.5 Flash) và tạo vector embedding hỗ trợ Lookalike search.	Áp dụng chính sách mặc định Last-Write-Wins hoặc ưu tiên đơn giản dựa trên cấu hình trait cơ bản.
Quyền riêng tư & AI làm giàu (Privacy & GenAI Enrichment)	Tự động phát hiện SHA-256 PII, sinh persona_name bằng LLM (Gemini 3.5 Flash) và tạo vector embedding hỗ trợ Lookalike search.	Cần cài đặt thêm các Function / Transformation tùy chỉnh bên ngoài pipeline chính để xử lý PII băm.
Chi phí Vận hành (Total Cost of Ownership - TCO)	Tối ưu chi phí hạ tầng (chỉ chi trả cho compute DB/Worker). Không phát sinh phí bản quyền theo số lượng hồ sơ (MTU).	Tính phí dựa trên số lượng người dùng theo dõi hàng tháng (Monthly Tracked Users - MTU). Chi phí tăng rất nhanh khi quy mô mở rộng.
Thời gian Kích hoạt (Ingestion Latency & Throughput)	Linh hoạt: Near Real-time (micro-batching với khóa FOR UPDATE NOWAIT) hoặc Batch định kỳ high-throughput.	Near Real-time theo kiến trúc streaming event-driven SaaS.

7.3 Tổng kết

Sự kết hợp giữa mô hình metadata-driven linh hoạt, khả năng xử lý PII băm bảo mật tích hợp GenAI, và cơ chế cách ly multi-tenant cấp CSDL giúp **Native C360 CIR Engine** trở thành giải pháp tối ưu cho doanh nghiệp muốn chủ động hoàn toàn về dữ liệu, đáp ứng tốt các bài toán hợp nhất danh tính phức tạp với chi phí TCO tối thiểu.